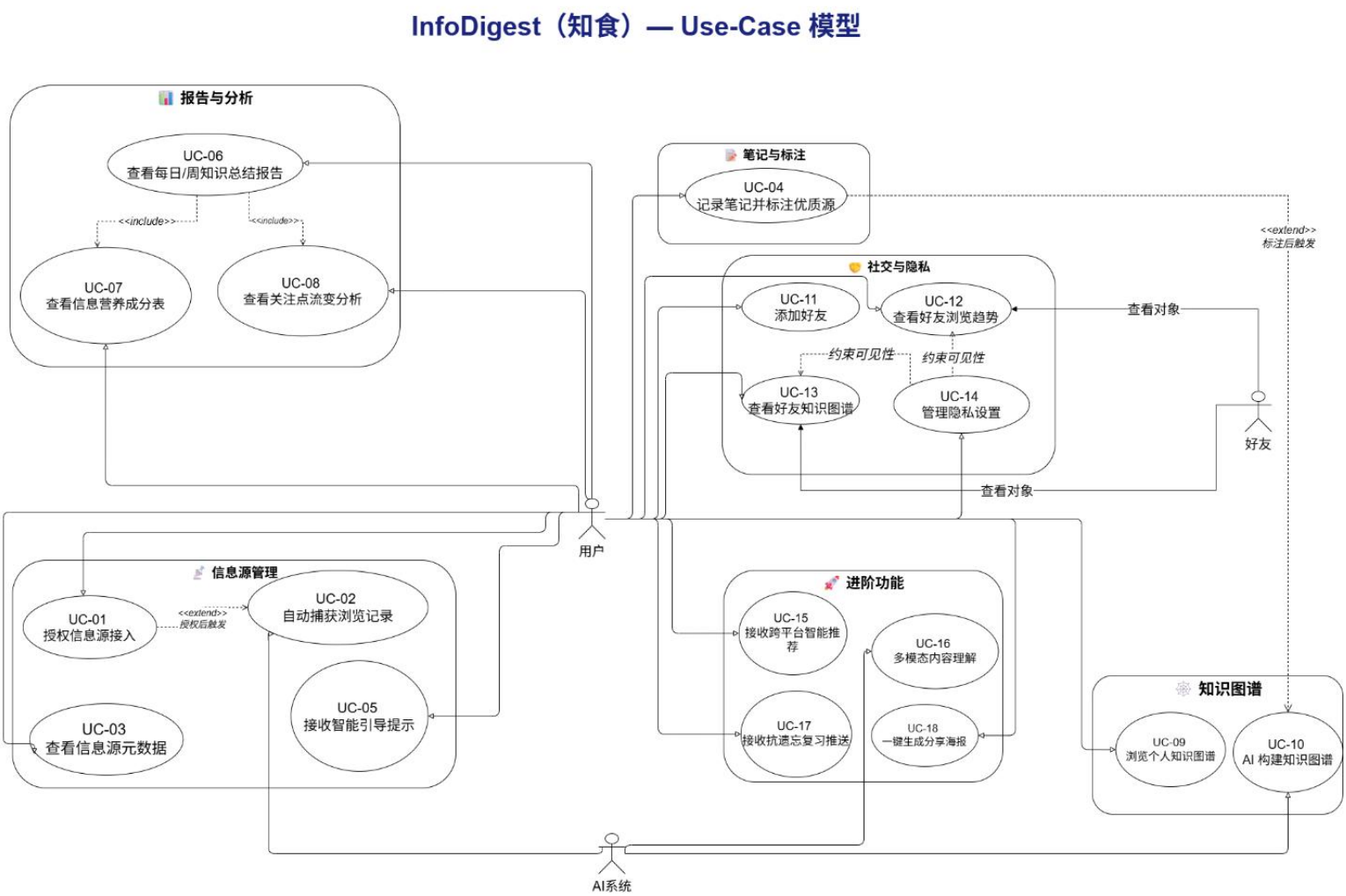
InfoDigest (知食) 系统 UML 模型设计文档

1. 用例模型 (Use Case Model)

本系统的用例架构按功能职责被划分为六大核心模块：报告与分析模块、信息源管理模块、笔记与标注模块、社交与隐私模块、进阶功能模块与知识图谱模块。这六大模块相互协作构成了完整的用户交互链路：负责价值洞察的报告与分析、负责内容获取的信息源管理、负责思考沉淀的笔记与标注、负责社交连接的社交与隐私、负责智能增强的进阶功能，以及负责知识网络构建的知识图谱模块。

1.1 总体用例模块与功能结构



(注：展示系统的模块划分，以及各模块中具有代表性的核心用例和交互关系。系统包含三个主要参与者：用户、好友以及 AI 系统。)

1.2 核心模块与架构用例分解

在整体框架中，报告与分析、知识图谱与进阶功能是本系统最关键的智能化能力。本节重点提取出对系统整体架构、技术栈选型与性能瓶颈产生重大影响的 **4 个核心用例场景**，并剖析其背后的底层流转与技术权衡。

1.2.1 隐式抓取与信息接入场景 (对应：信息源管理模块)

- **核心用例**：自动捕获浏览记录。
- **架构驱动意义**：此用例是全站数据流的源头，直接驱动了后端 Crawler Engine（爬虫引擎）和 Scheduler（定时调度任务）的架构设计。由于外部第三方数据（如 B站、知乎）的格式高度异构，该用例迫使系统在持久层摒弃传统的严格列字段设计，采用 MySQL 的 JSON 扩展字段来弹性存储平台元数据。

1.2.2 AI 营养分析与图谱构建场景 (对应：报告与分析 / 知识图谱模块)

- **核心用例**：查看信息营养成分表 / AI 构建知识图谱。
- **架构驱动意义**：这是全站算力与 I/O 的最大瓶颈。由于外部大语言模型（LLM）API 调用耗时极长，该用例强制要求后端架构必须采用**异步非阻塞**的处理管线；同时，系统深度依赖大模型抽取实体（如“多巴胺指数”），这驱动了在业务层必须引入严格的 JSON 反序列化与幂等容错机制，以防止大模型产生的“幻觉”导致脏数据。

1.2.3 知识拓扑交互与呈现场景 (对应：知识图谱模块)

- **核心用例**：浏览个人知识图谱。
- **架构驱动意义**：决定了前端表现层的技术选型。为了实现成百上千个节点的网状拖拽、缩放等高性能交互，该用例驱动前端引入了 React Flow 与 ECharts 渲染引擎。基于轻量化部署考量，系统放弃了厚重的 Neo4j 图数据库，转而要求后端通过 MySQL 的邻接表（Node/Edge 表）高效组装图谱 JSON 结构树。

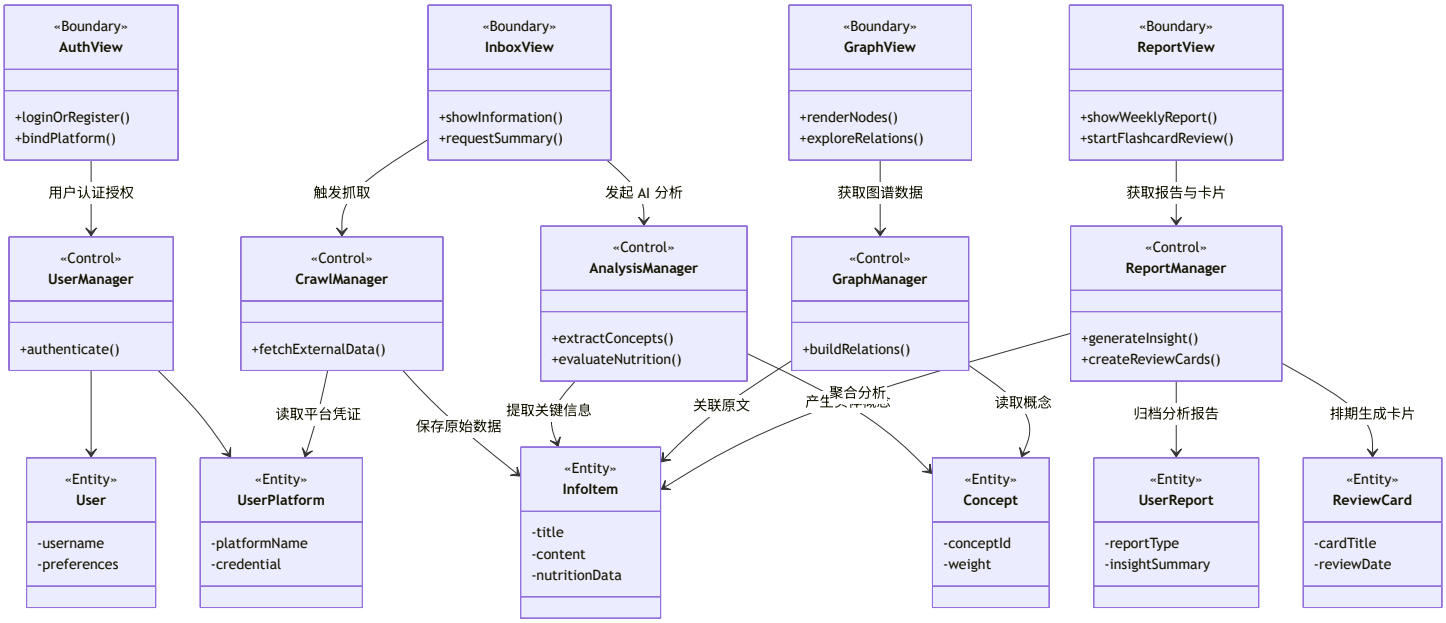
1.2.4 细粒度隐私网关管控场景 (对应：社交与隐私模块)

- **核心用例**：管理隐私设置 / 查看好友知识图谱。
- **架构驱动意义**：决定了系统的核心鉴权与安全架构。它不仅要求实现 JWT 身份校验，更要求在 Spring Boot 的业务逻辑层（Service）和数据库查询层切入全局的隐私拦截过滤（Data Masking）。当组装好友图谱数据时，必须在 SQL 层面实时剔除标记为“私密”的节点和边，涉及高频连表查询的性能优化。

2. 分析模型 (Analysis Model)

本分析模型采用经典的 BCE（边界-控制-实体）架构模式，从全局视角展示了系统各核心功能的内部概念运行逻辑，剥离了底层技术细节。

2.1 全局分析类图



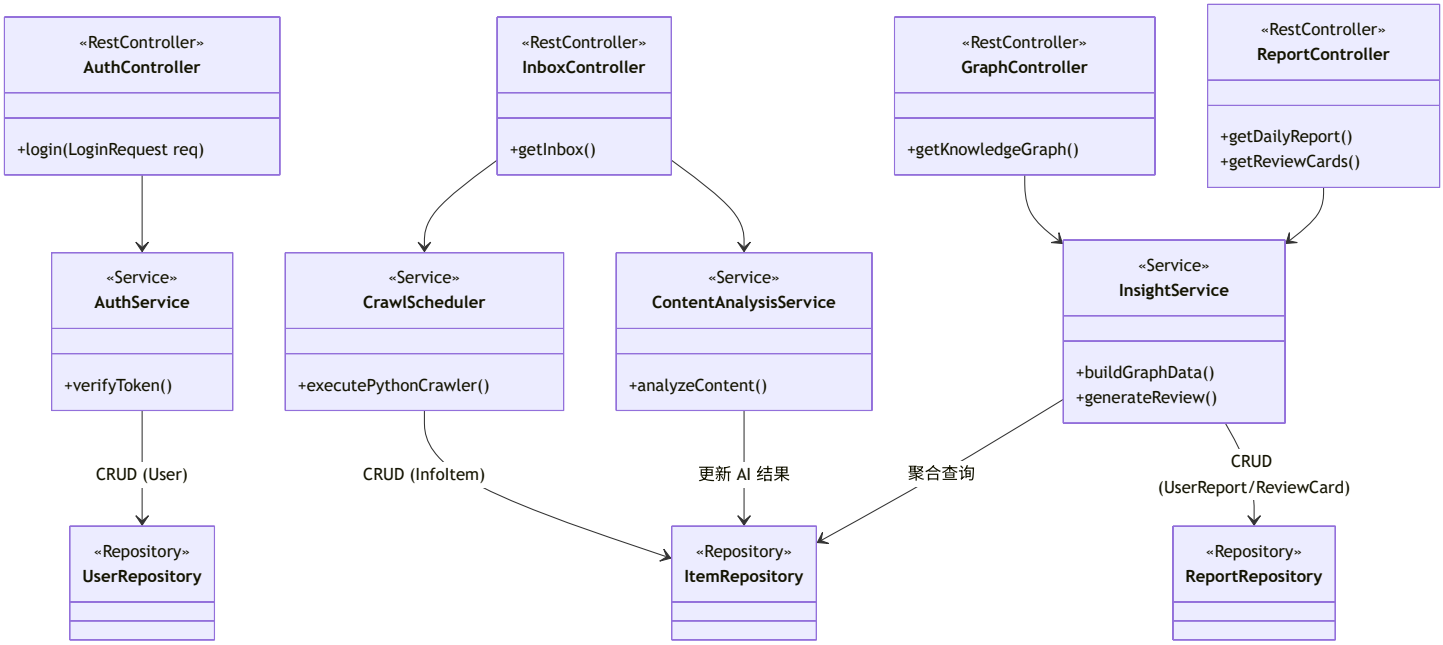
2.2 分析模型说明

整个系统通过 **边界类** 接收前端四大核心界面的指令。对应的 **控制类** 会承接这些指令，例如 `AnalysisManager` 统筹大语言模型的处理逻辑， `CrawlManager` 统筹爬虫脚本的调用逻辑。最后，控制层会对 `InfoItem` 、 `Concept` 和 `ReviewCard` 等核心 **实体类** 的状态进行读写，完成信息的闭环流转。

3. 设计模型 (Design Model)

全局设计模型是将上述分析模型映射到实际的 **Spring Boot 后端架构**中的体现。边界类被具象化为 HTTP 接口控制器，控制类演变为依赖注入的服务（Service），同时引入了数据仓库层（Repository）来实现实体的持久化。

3.1 全局设计类图



3.2 设计模型说明

在实际的代码工程设计中，系统采用了严格的分层架构设计：

1. **接口解耦**：所有的外部请求入口集中在 `Controller` 层，直接映射了各种 RESTful API 端点。
2. **核心业务下沉**：由 `ContentAnalysisService` 和 `InsightService` 承担最重的计算任务（包括大模型 API 调用、图谱节点计算、艾宾浩斯记忆卡片排期等）。
3. **集中持久化**：所有的数据库操作被收拢在统一的 `Repository` 层中，利用 Spring Data JPA 技术框架，业务层不再直接操作复杂的 SQL 语句，实现了数据驱动模型的安全落地。